# Data Driven Control Integration Week

## Group 8 (MPC)

Christian Brunet  Jules Marceau
Matthieu Moussaoui  Clément Tamboise
Rayane Trigoust

CentraleSupélec

Friday, November 6, 2026

# Table of Contents


1. Introduction
2. Dataset Validation
3. System Modeling
4. First RNN
5. Z-axis control

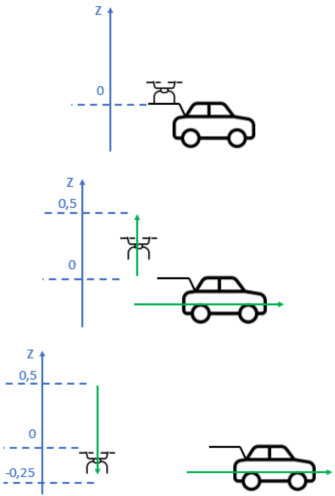
# Project Objectives

**Available Resources:**

- Dataset of vertical acceleration.
- MATLAB/Simulink drone simulator.

**Objectives:**

- Validate the dataset.
- Identify the model.
- Develop a control law.



# Dataset Validation

## Cleaning the data

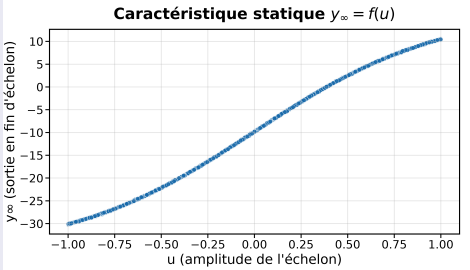
**Remove lines that includes:**

- Frozen sensor for 2+ seconds
- 5+ diverging values

## Normalisation

- Directly in the training

## Static characteristic




$y_\infty = f(u)$ after eliminating corrupted runs.

# System Modeling

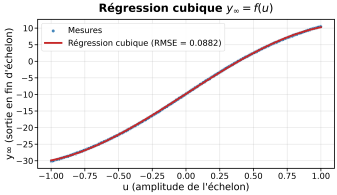
## Linear Model: Hammerstein Approach

### Static Characteristic $N(u)$

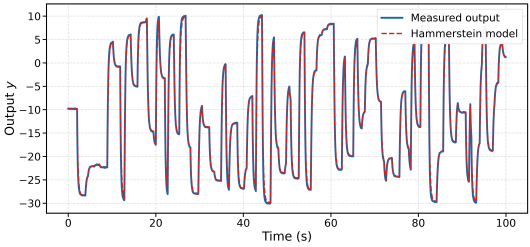
3rd-order polynomial identified on steady-state regimes:

$$v(t) = N(u) = a_3 u^3 + a_2 u^2 + a_1 u + a_0$$

**Tau identification**




**Hammerstein Model Validation**




**Least-Squares Linear Model Validation (a=0.7836, b=4.9676, c=-2.1222)**

# System Modeling

## Linear Model: Least Square Method

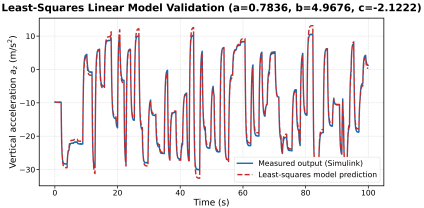
### Model equations

$$y_k(t+1) = a\,y_k(t) + b\,u_k(t) + c$$
$$= \varphi_k^{\mathsf{T}}\theta.$$

$$\varphi_k^{\mathsf{T}} = \begin{bmatrix} y_k(t) & u_k(t) & 1 \end{bmatrix} \qquad \theta = \begin{bmatrix} a & b & c \end{bmatrix}^{\mathsf{T}}.$$

The least-squares estimate is

$$\theta^{\star} = \left(\Phi^{\mathsf{T}}\Phi\right)^{-1}\Phi^{\mathsf{T}}Y.$$

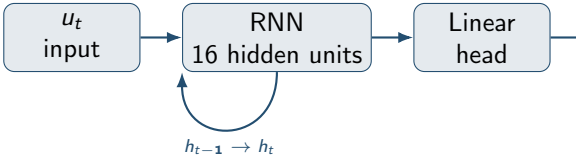


# First RNN

## Sequence-to-sequence architecture

### Parameters

- One scalar input $u_t$ and one scalar output $\hat{y}_t$
- Hidden state dimension: 16
- Early stopping to avoid overfitting




### Forward equations

$$h_t = \tanh(W_{ih}u_t + W_{hh}h_{t-1} + b)$$

$$\hat{y}_t = W_{hy}h_t + b_y$$

### Least-squares criterion

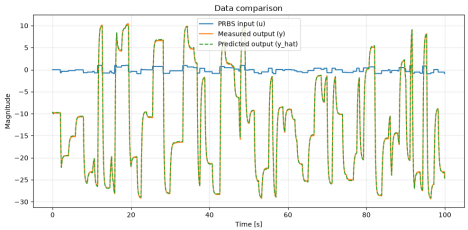
$$\sum^{N}$$

# First RNN

## Results

### Numerical results

- Validation loss: $3.93 \times 10^{-4}$
- Test loss: $4.01 \times 10^{-4}$
- Training completed at epoch 100
- Only 321 trainable parameters



# Z-axis control

## Designing the command

### Principle

- Python Class Wrapping
- Simulink Signal Generation
- LQ control for the linear model
- MPC control for the RNN

### Results

Add here the main result or performance metric.

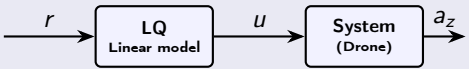
### Figure

*Insert figure here*

# LQ control

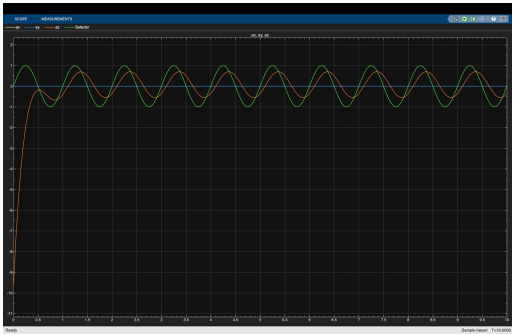
## Linear-quadratic control design

### Control principle




### Performance

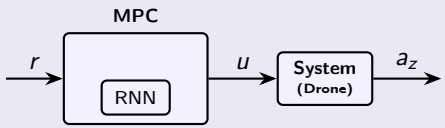
- Global trajectory following
- Wrong amplitude + phase shift
- Missing prediction



# MPC control

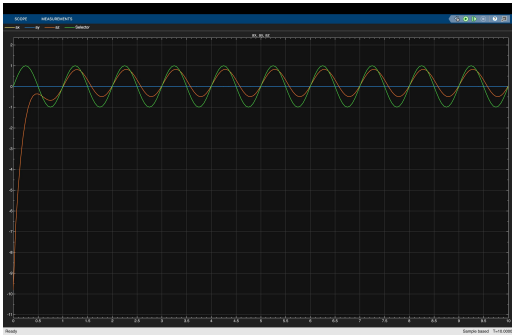
## Model predictive control design

### Control principle




### Performance

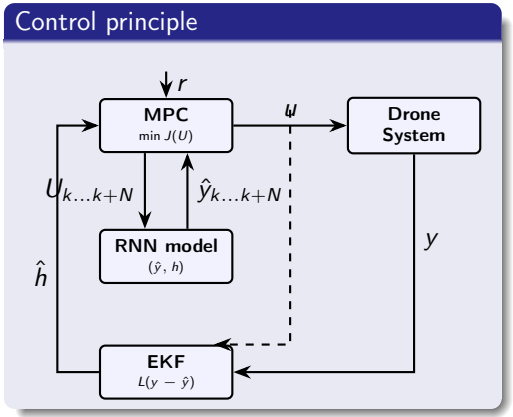
- Good overall tracking.
- No phase shifting good prediction.
- Full amplitude not reached.
- Only internal state.

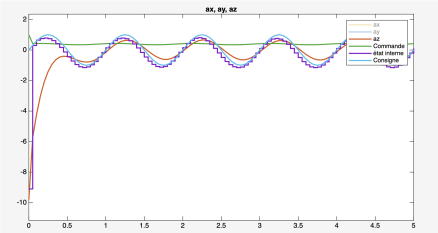


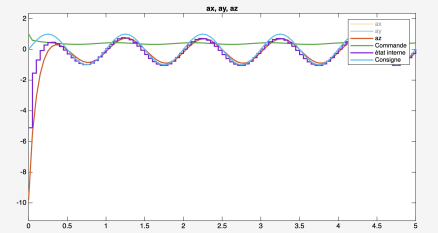
# MPC with EKF

## State estimation and predictive control

### Control principle



# Requested command

MPC control with EKF feedback

## Control objective

Replace this text with the analysis of the requested command:

- Describe the reference trajectory.
- Explain how the EKF estimate is used by the MPC.
- Discuss tracking accuracy and transient behavior.

## Performance

Add here the main tracking result or metric.

*Insert requested-command figure here*

# Comparison

## LQ, MPC and MPC with EKF

### LQ control

- Simple and computationally efficient.
- Based on a linear model.
- No state measurement feedback.

### MPC control

- Uses a predictive RNN model.
- Handles nonlinear dynamics.
- No EKF state correction.

### MPC with EKF

- Corrected state estimation.
- Feedback from measurements.
- Improved robustness.